Case	Original code snippet	New code snippet that creates bugs	Change explained	Initial reflection	AI's reflection label	AI's guidance	New reflection
1	num_users = 500 num_videos = 1000 num_samples = 10000 num_categories = 5	num_users = 250 num_videos = 500 num_samples = 5000 num_categories = 5	Within yt_data_generator.py, I halved each of the parameters that go into generating the csv file for data that the AI trains on.	After updating yt_model_trainer_torch.py with the csv generated with the modified parameters, I ran it and was surprised to see that the results and graph were fairly similar to that of the original csv runs. I assumed that less training data would result in a drastic decrease in accuracy, but this was proven untrue. Even when halving the number of epochs from 60 to 30, the loss and accuracy were the same.	Bad	You're describing what happened, but not really digging into why it happened. Right now your reasoning is basically: "less data → should be worse → but it wasn't," and then you stop there. Let me push your thinking a bit: ⚠Are you sure halving from 10,000 to 5,000 samples is actually small enough to hurt performance for this specific model? ⚠What does your dataset look like—could it already be simple or repetitive enough that even 5,000 samples is plenty? ⚠Did you check whether your model might already be underfitting or overfitting before the change? ⚠Are your training and test sets truly independent, or could something like data leakage be making accuracy look artificially stable? ⚠When you say "results were similar," how similar—are you comparing exact metrics, trends, or just visual impressions? Try to move from: "This is surprising" to: "Here are 2–3 concrete hypotheses for why reducing the dataset didn't change performance, based on how the model and data behave."	Thinking deeper, the original midterm project had me deploy a simple idea with a simple execution. The accuracy when running was already around 60, and back then when I had investigated, this was suposedly a good value to be around. Probing further now, it is possible that 10000 samples was a lot to begin with, and perhaps the amount of parameters and variables was not enough to convolute the training process enough to see a change in results. I went back to my code to see what would happen if I continued to tweak the parameter values, and observed that a number of samples in the hundreds actually increased the accuracy from around 62-64 to around 67. So with a little more tweaking, my original idea that decreasing the parameters would decrease accuracy was actually proven incorrect. I think this is because scaling down all parameters, instead of tweaking a single parameter, only scales down the results. The accuracy is still the same, just on smaller numbers. I think for the rest of my self-created bugs, I am going to make more drastic changes.
2	import tensorflow as tf from tensorflow.keras.models import Sequential from tensorflow.keras.layers import Dense from tensorflow.keras.callbacks import EarlyStopping from tensorflow.keras.layers import Input		The trainer runs really slowly, and gets stuck on the first epoch. This is without any modification to the file.	All the tensorflow imports have yellow squiggly lines under them, due to a missing module error. Back when I first generated this code, I downloaded the necessary packs through terminal. I originally thought that since I moved the files around to put them in a folder, that I just had to redownload the package, but it said "Requirements already satisfied:..." When trying to train it with the modified, smaller data set, it still got stuck on the first epoch, so because of that, it is not the size of the data causing the error. Sequential and Input are white text, even though they are both used to build the feedforward neural network. Dense and EarlyStopping do not have this issue.	Bad	Right now you're focusing on surface signals (yellow squiggly lines, reinstalling packages, text color in the editor) rather than the actual behavior of the system. Let me push your thinking: ⚠Do editor warnings (like squiggly lines) always mean the code will fail at runtime? Or could they be unrelated to the slowdown? ⚠If the program is running but stuck/slow, what does that suggest compared to a true "missing module" error? ⚠What exactly does "stuck on the first epoch" mean—CPU at 100%, no output, extremely long step time? What is the program actually doing during that time? ⚠Did anything about your data pipeline change (file size, format, loading method) that could affect performance rather than correctness? ⚠Are you sure the issue is in the imports, or could it be somewhere else in the training loop or data feeding process? ⚠How could you isolate whether the slowdown comes from: ⚠data loading ⚠model computation ⚠or environment configuration? Try to shift from: "There are warnings, so that must be the problem" to: "The system runs but behaves slowly—what part of the pipeline is causing the bottleneck, and what evidence supports that?"	When I first saw the yellow squiggly lines under the TensorFlow imports, I assumed that was the reason the trainer was running so slowly and getting stuck on the first epoch. I even tried reinstalling the packages, but nothing changed. What confused me was that the script didn't crash, but it barely moved past the first epoch and looked frozen. After thinking about it more, I realized I was focusing too much on the editor warnings instead of what the program was actually doing. If the imports were truly broken, the script wouldn't run at all. The fact that it did run means the slowdown had to come from somewhere else. Looking closer, "stuck on the first epoch" probably didn't mean TensorFlow was broken, it just meant that each step inside that epoch was taking a long time. That could be from the data loading, the environment configuration, or even how many threads TensorFlow was allowed to use. I had moved files around and changed my setup earlier, so it's possible something in the environment changed without me noticing. This bug helped me realize that slow behavior isn't the same as a missing module error. Instead of assuming the warnings were the cause, I needed to think about what part of the training loop was actually doing the work and why it might be taking longer than before.
3	X = data.drop("clicked", axis=1)	X = data.drop(["clicked", "length_similarity"], axis=1)	In yt_model_trainer.py, I removed the length_similarity feature from the training data.	When I removed the length_similarity feature, I expected the model to maybe struggle a little, but I didn't think the accuracy would drop as much as it did. At first I thought the model was just having a bad run, but then I remembered that the data generator actually uses this feature directly when computing the click probability. So by taking it away, I basically hid one of the strongest signals the model relies on. The model still trained and produced results, but they were noticeably worse. This made me realize that even small changes to the feature set can have a big impact when the generator and the model are tightly connected. I changed the input without changing the underlying logic, so the model was trying to learn a pattern it no longer had access to.	Good	This shows you understand: ⚠the relationship between features and labels ⚠and why the model's performance depends on having access to key predictive signals Why this is strong You're not just saying "accuracy dropped" — you're explaining: ⚠where the signal comes from ⚠why removing it hurts learning ⚠and how the model is now missing critical information That's exactly the kind of reasoning your instructor is looking for. One small push (not required, just to make it even stronger) You could deepen it slightly by asking: ⚠Was this feature redundant with others, or uniquely important? ⚠Does the drop suggest the model was heavily dependent on a single feature? But even without that, this is already A-level quality. Send the next one when you're ready 👍	
4	clicked = np.random.binomial(1, probability)	clicked = 1 - np.random.binomial(1, probability)	I inverted the click outcome in the data generator, thinking it would just "flip" the labels but still be learnable.	The model still trains and runs, but accuracy hovers around ~0.5 and the confusion matrix shows almost random predictions. I didn't realize that by flipping the labels without changing the features or the probability function, I essentially made the model learn the opposite of what the generator was designed for. The model architecture and training code are fine, but the semantics of "1 = clicked" vs "0 = not clicked" are now reversed, and I never updated my interpretation. When I flipped the click labels, I expected the model to still learn something, just the opposite pattern. Instead, the accuracy dropped to around 0.5 and the predictions looked almost random. At first this confused me, because the model architecture didn't change at all. But then I realized that the generator still computes the click probability the same way, and I only changed the final label. So the features still point toward the original meaning of "clicked," but the labels now mean the opposite. The model is basically being asked to learn a backwards version of the data without any hint that the meaning changed. This helped me understand how important label semantics are, since the model can't learn the right thing if the labels no longer match the logic that produced them.	Bad	You're circling around something important, but your core conclusion is off. You correctly noticed: ⚠labels were flipped ⚠performance dropped to ~0.5 ⚠predictions look random But this reasoning is shaky: "the model is being asked to learn a backwards version without any hint" That's not quite right. If everything is consistent, learning the "opposite" mapping should still be just as learnable. Let me push your thinking: ⚠If you perfectly flip all labels, is the problem actually harder—or just inverted? ⚠Would a model trained on flipped labels still be able to achieve high accuracy (just predicting the opposite class)? ⚠If the answer is yes, then why are you seeing ~0.5 instead of something like ~0.6–0.7? Now ask yourself: ⚠Is the flip happening cleanly and consistently, or is there some added randomness? ⚠What exactly does np.random.binomial(1, probability) represent—and what happens when you apply 1 - (...) to it? ⚠Are you flipping the probability, or flipping a sampled random outcome? ⚠Could this introduce extra noise instead of a clean inversion? Where to go next Try to move from: "labels were reversed so the model got confused" to something more precise like: "the transformation I applied may have changed the statistical relationship between features and labels in a non-trivial way" You're close—but you need to reconcile: 👉 why a theoretically learnable flipped mapping ends up looking random in practice	When I flipped the click labels, I expected the model to still be able to learn a clear pattern, just the opposite one. In theory, if you cleanly invert all labels, the problem shouldn't become harder, it should just swap what counts as "clicked" and "not clicked," and a good model could still reach similar accuracy. But what I actually saw was accuracy dropping to around 0.5 and the predictions looking almost random. That made me realize I wasn't just inverting a clean mapping; I was inverting the sampled outcomes from np.random.binomial(1, probability), not the underlying probabilities themselves. The generator still uses the same score function to produce click probabilities, but by applying 1 - (...) to the sampled clicks, I'm effectively adding extra randomness on top of an already stochastic process. Instead of a neat "opposite" relationship, I've weakened the statistical link between the features and the labels. So the model isn't just learning a reversed pattern, it's trying to learn from labels that now have more noise and less signal, which explains why performance collapses toward random guessing.

5	optimizer = optim.Adam(model.parameters(), lr=0.001)	optimizer = optim.Adam(model.parameters(), lr=1.0)	In yt_model_trainer_torch.py, I drastically increased the learning rate to see if training would be faster.	I increased the learning rate thinking it would just make training faster, but instead the loss immediately blew up and turned into NaN. At first I thought something else in the code broke, because I didn't expect such a dramatic reaction from changing a single number. After looking at it more closely, I realized I completely misunderstood how sensitive the optimizer is to the learning rate. With such a large value, the model overshoots the minimum so aggressively that the weights explode, and the loss becomes impossible to compute. This bug made me appreciate how delicate training can be, because even though the model and data were fine, one hyperparameter pushed everything off the rails.	Good	This is a strong reflection. You clearly: ✅ Identified the specific change (learning rate from 0.001 → 1.0) ✅ Described the observed behavior (loss → NaN) ✅ Corrected your initial misconception ("faster training") ✅ And gave a mechanistic explanation (overshooting → exploding weights → invalid loss) Why this works well Your explanation connects: ✅ learning rate → update size ✅ update size → instability ✅ instability → numerical breakdown (NaN) That's exactly the kind of causal chain your instructor is looking for. One small refinement (optional, to make it even sharper) You might push it slightly further by questioning: ❓ Why does overshooting lead specifically to NaN? ❓ What operations in the model/loss could become numerically unstable when values grow too large? But even without that, this is already clear, accurate, and well-reasoned. Send the next one 👍	
6	self.model = nn.Sequential(nn.Linear(input_dim, 64), nn.ReLU(), nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 1), nn.Sigmoid()) criterion = nn.BCELoss()	self.model = nn.Sequential(nn.Linear(input_dim, 64), nn.ReLU(), nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 1)) criterion = nn.BCELoss()	I removed a training layer, sigmoid, in the _torch file to see how the graph would be affected.	When I removed the sigmoid layer, I expected the model to still run, maybe with weird accuracy or a messy loss curve. Instead, it immediately threw an error saying that all inputs to BCELoss must be between 0 and 1. At first this confused me, because I thought the loss function would just "handle" whatever the model output. Thinking about it more, I realized I broke the basic assumption the loss function relies on. Without the sigmoid, the model outputs raw values that can be negative or greater than 1, so BCELoss refuses to even start training. This bug made me realize how tightly connected the architecture and loss function are. I changed one piece without thinking about the other, and the whole thing fell apart before training even began.	Good	This is a strong reflection. You: ✅ Clearly identified the exact change (removing Sigmoid) ✅ Described the unexpected behavior (runtime error from BCELoss) ✅ Corrected your misconception about the loss function "handling anything" ✅ And most importantly, connected it to a violated assumption (inputs must be in [0,1]) Why this stands out You explicitly recognized: the architecture and loss function are tightly coupled That's a key deep learning concept, and you explained it correctly: ✅ Sigmoid → constrains outputs to probabilities ✅ BCELoss → expects probabilities ✅ removing it → invalid input domain → failure before training Small refinement (optional, for even deeper insight) You could push one step further by asking: ❓ Why does BCELoss enforce this constraint instead of internally applying a sigmoid? ❓ What does that imply about how losses are designed vs. model outputs? But that's just extra depth—what you have is already clear, accurate, and conceptually strong. Send the next one 👍	
7	X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)	X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) np.random.shuffle(y_train)	I shuffled the labels in the trainer, thinking it might test robustness.	When I shuffled only the training labels, I expected the accuracy to drop, but I didn't expect it to collapse so completely. The model trained without errors, but the test accuracy hovered around 0.5 and the predictions looked random. Thinking about it more, this makes perfect sense: I broke the relationship between the features and the labels in the training set. The model was basically learning noise, while the test set still had the correct labels. This bug helped me see how fragile the learning process is. Even a small amount of label noise can ruin everything, and in this case I introduced the worst possible version of it.	Good	This is a strong reflection. You clearly: ✅ Identified the exact bug (shuffling only y_train) ✅ Described the observed behavior (accuracy ≈ 0.5, random predictions) ✅ And correctly explained the core issue: the relationship between features and labels was broken Why this is solid You recognized that: ✅ training data became pure noise ✅ test data remained correct ✅ so the model had no learnable signal That's exactly the right causal chain. Your statement: "the model was basically learning noise" is accurate and well-applied here. One small push (to make it even stronger) This part is slightly off: "even a small amount of label noise can ruin everything, and in this case I introduced the worst possible version of it" Ask yourself: ❓ Did you introduce a small amount of noise—or something more extreme? ❓ What's the difference between partial label noise vs complete randomization? Refining that distinction would make your explanation sharper. Overall, though—this is clear, correct, and well-reasoned.	
8	history = model.fit(X_train, y_train, validation_split=0.2, epochs=25, batch_size=32, callbacks=[early_stop], verbose=1)	history = model.fit(X_train, y_train, validation_split=0.2, epochs=25, batch_size=100000, callbacks=[early_stop], verbose=1)	I set the batch size to a huge number, thinking it would just use the whole dataset as one batch.	I set the batch size to a huge number thinking it would just treat the whole dataset as one batch. Instead, training became extremely slow and felt like it was stuck. At first I thought TensorFlow was freezing, but then I realized it was trying to process everything at once in a way that wasn't efficient at all. I didn't consider how batch size interacts with hardware limits or how it affects the number of steps per epoch. This bug made me realize that training configuration directly affects how the model uses resources. Even though nothing technically broke, the model was running in a way that made training almost unusable.	Bad	You're on the right track noticing that training behavior changed, but your explanation is still a bit hand-wavy and misses the core mechanics. Let me push your thinking: ❓ If your dataset has fewer than 100,000 samples, what does setting batch_size=100000 actually do? Does it really create something "inefficient," or just reduce the number of steps per epoch? ❓ How many steps per epoch are you actually running now compared to before? What does that imply about how often weights get updated? ❓ Would you expect fewer updates per epoch to make training feel slower, even if each step processes more data? ❓ Is the slowdown coming from computation per step, or from how progress is being reported (e.g., fewer visible updates/logs)? ❓ How could you verify whether it's truly "slow" vs. just appearing stuck? Try to move from: "it was inefficient and used too many resources" to: "this change altered the number of gradient updates and how work is distributed per epoch, which affected the perceived training behavior in a specific, measurable way" Right now, your explanation leans on intuition about hardware rather than analyzing what the training loop is actually doing.	When I set the batch size to 100,000, I originally thought it would just treat the whole dataset as one big batch. Instead, training looked like it was stuck, barely printing anything, and it felt way slower than before. At first I assumed TensorFlow was freezing or doing something inefficient, but after thinking about it more, I realized the behavior actually makes sense. My dataset doesn't even have 100,000 samples, so by setting the batch size that high, I accidentally reduced the number of steps per epoch to just one. That means the model only updates its weights once per epoch instead of many times, so there's a lot less visible progress during training. The "slowness" was the training loop doing far fewer updates and giving me fewer logs to look at. This bug helped me understand that batch size doesn't just affect how much data is processed at once; it also changes how often the model learns and how training progress is reported. Even though nothing technically broke, the training process felt completely different because I changed how the work was distributed across each epoch.

9	scaler = StandardScaler() X_train = scaler.fit_transform(X_train) X_test = scaler.transform(X_test)	I removed this	In _torch, I removed scaling entirely to see what effects it would have on the graph.	When I removed the standardization step, I expected the model to still learn, just maybe a little slower. The model did run, but the loss curve was much less stable and the final accuracy was noticeably worse. At first I thought the model architecture was the problem, but then I remembered that the features in this dataset are on very different scales. Without scaling, some features dominate the gradients while others barely matter. This bug helped me understand why preprocessing is such a big deal. Even though the code still works, the model is learning from inputs that aren't balanced, and that makes training much harder than it needs to be.	Good	This is a strong reflection. You: ✅ clearly stated the change (removing standardization) ✅ described both training behavior (unstable loss) and outcome (worse accuracy) ✅ and gave a mechanistic explanation tied to feature scales and gradients Why this works You correctly connected: ✅ different feature scales → uneven gradient contributions ✅ → training instability ✅ → poorer convergence That's exactly the right chain of reasoning for why scaling matters. Small refinement (optional, for extra depth) You could sharpen it by asking: ✅ Which features might dominate, and how would that affect the direction of updates? ✅ Why would instability show up in the loss curve specifically (e.g., oscillations vs slow convergence)? But overall, this is clear, accurate, and conceptually solid—A-level reasoning. Send the next one
10	data = pd.read_csv("youtube_recommendation_dataset.csv")	data = pd.read_csv("youtube_recommendation_dataset.csv") data["user_avg_watch_ratio"] = 0.5 data["user_avg_video_length"] = 10 data["user_activity_level"] = 0.7	I replaced all user-specific features with constant values, removing all variation between users.	This bug was interesting because nothing crashed, but the results were noticeably worse. I set all the user-related features to constant values, thinking it would just "simplify" the dataset. Instead, the accuracy dropped and the model basically predicted the same thing for every sample. Looking back, it makes sense: I removed all the variation that tells the model how different users behave. The generator still uses those features to compute the click probability, but the model no longer sees any of that information. So even though everything technically ran fine, the model was learning from a crippled dataset. This helped me understand how easy it is to introduce silent data corruption that doesn't throw errors but completely changes what the model is capable of learning.	Good	This is a strong reflection. You clearly: ✅ identified the exact change (making user features constant) ✅ described the behavior (model trains, but worse accuracy and near-constant predictions) ✅ and connected it to the loss of informative variation Why this works You correctly reasoned that: ✅ those features encode user-specific signal ✅ making them constant removes useful information ✅ the generator still depends on them → so the model is missing part of the true signal Your conclusion about "silent data corruption" is especially strong—that's exactly the kind of real-world issue this project is meant to highlight. Small refinement (optional, for even deeper insight) You could push one step further by asking: ✅ If user features are constant, what remaining features is the model relying on? ✅ Does the near-constant prediction suggest class imbalance or reduced separability? But overall, this is clear, accurate, and insightful—A-level reasoning. Send the next one